

DEVELOPER GUIDE · V1.0

SAR → EO GAN Pipeline

A complete guide for software developers — from zero knowledge to running a production-grade satellite image translation pipeline.

June 2026

16 Chapters · Setup · Models · Training · Tools

Written for developers new to deep learning pipelines

Table of Contents

01 What Is This Project and Why Does It Exist?

[What is SAR? What is EO?](#)

[What is a GAN?](#)

[What does pix2pix do?](#)

02 Manual Project Setup from Scratch

[Prerequisites](#)

[Clone and environment](#)

[Install dependencies](#)

[Verify installation](#)

03 Project Structure — What Every File Does

04 Configuration System (Hydra)

05 Data Pipeline — From Raw Satellite to Training Tensors

[Raw Sentinel data](#)

[Preprocessing \(sentinel_utils.py\)](#)

[Dataset classes](#)

[Augmentation](#)

[Weighted Random Sampling](#)

06 Models — Architecture Deep Dive

[U-Net Generator](#)

[PatchGAN Discriminator](#)

[Multi-Scale Discriminator](#)

[DCGAN Models](#)

07 Loss Functions — What the Model Optimises and Why

08 Trainers — The Training Loop Explained

09 Makefile — Developer Shortcuts

10 Pre-commit Hooks — Automated Quality Gates

11 CI/CD Workflows (GitHub Actions)

12 MLflow — Experiment Tracking

13	Docker — Reproducible Environments	
14	Claude Code Skills — AI Workflow Automation	
15	Testing — How the Codebase Verifies Itself	
16	End-to-End Workflow — First Training Run	
17	How to Build a Custom Dataset	PyTorch dataset contract, SideBySidePairedDataset anatomy, HDF5 example, synchronized augmentation
18	How to Build a Custom Model	Base class contract, ResNet Generator step by step, config registration, tests
19	How to Build a Custom Loss Function	Loss enum, Relativistic Hinge, LPIPS perceptual loss
20	How to Build a New Training Workflow	Trainer skeleton, CycleGAN unpaired workflow end to end
21	How the CI/CD and Pre-commit Were Built	Pre-commit from scratch, GitHub Actions from scratch, MLflow integration, pyproject.toml packaging
22	Component Integration Map — How Everything Connects	Full call chain, tensor shape flow, config dependency map, output file origins
23	Managing pyproject.toml from the Command Line	uv (recommended), Poetry, pip-tools — initialise, add, remove, upgrade dependencies without editing TOML by hand
24	Managing CI/CD from the Command Line	gh CLI, act (local runs), pre-commit CLI, secrets, workflow triggers — full CI/CD control without the GitHub UI

What Is This Project and Why Does It Exist?

WHO THIS IS FOR

You are a software developer who has been handed a task: get this GAN pipeline running. You may not know what SAR is, what a GAN is, or what pix2pix does. This chapter explains all three from scratch before you touch a single line of code.

1.1 What is SAR? What is EO?

Satellites orbit the Earth and take pictures. There are two major types of imaging sensors:

SAR — Synthetic Aperture Radar

Emits microwave radio pulses and measures what bounces back. Works through clouds, rain, and darkness. The images look strange to humans — bright areas are rough or metallic surfaces, dark areas are smooth water or pavement.

Satellite: Sentinel-1 (ESA)

EO — Electro-Optical (Optical)

Captures visible light like a camera. Produces natural-looking images humans understand intuitively. But it cannot see through clouds, and it only works in daylight.

Satellite: Sentinel-2 (ESA)

The problem: most of Earth's surface is covered by clouds most of the time. SAR sees everything but is hard to interpret. Optical images are intuitive but often unavailable. **The goal of this pipeline is to train a model that takes a SAR image as input and outputs a realistic-looking optical image** — giving analysts the best of both worlds.

1.2 What is a GAN?

A **Generative Adversarial Network (GAN)** is a machine learning architecture consisting of two neural networks that compete against each other:

G

Generator — takes an input and tries to produce a realistic-looking output. It starts knowing nothing and gradually learns to produce convincing images.

D

Discriminator — looks at images and tries to tell whether they are real (from the dataset) or fake (produced by the Generator). It also starts knowing nothing.

They train together in a loop. The Generator gets better at fooling the Discriminator. The Discriminator gets better at spotting fakes. Eventually the Generator produces images realistic enough that the Discriminator cannot reliably tell them apart from real ones. At that point, the Generator is useful.

ANALOGY

Think of the Generator as a counterfeiter and the Discriminator as a detective. The counterfeiter keeps improving their forgeries, the detective keeps improving their detection skills. After enough iterations, the counterfeiter is so good that the detective has to guess randomly — meaning the forgeries are indistinguishable from real currency.

1.3 What does pix2pix do?

pix2pix is a specific GAN architecture for image-to-image translation — transforming one type of image into another. Unlike a basic GAN that generates images from random noise, pix2pix takes a conditioning input (the SAR image) and generates an output conditioned on that input (the corresponding EO image).

It requires **paired training data**: for every SAR image, you need the matching optical image of the same location at the same time. The Sentinel-1 and Sentinel-2 satellites are both operated by ESA and their images can be co-registered (aligned) to create these pairs.

```
Training pair: [ SAR image ] → Generator → [ Fake EO image ]
               |
               |
[ Real EO image ] —————|
                           |
                           ↓
                   Discriminator
                   (real or fake?)
```

Manual Project Setup from Scratch

NOTE

This chapter covers manual setup step by step. Every command is explained — not just what to type but why you are typing it.

2.1 Prerequisites

Before starting, you need these tools installed on your machine:

Tool	Why you need it	Check it exists
<code>Python 3.10+</code>	The pipeline uses type hints and syntax introduced in 3.10	<code>python3 --version</code>
<code>git</code>	To clone the repository and manage commits	<code>git --version</code>
<code>pip</code>	Python package installer (comes with Python)	<code>pip --version</code>
8 GB RAM minimum	PyTorch loads large model weights into memory	Check system info
GPU with CUDA (optional)	Training is 20–50× faster on GPU; CPU is fine for testing	<code>nvidia-smi</code>

2.2 Clone and create a virtual environment

Step 1 — Clone the repository

```
git clone https://github.com/baudhya/gan-pipeline.git
cd gan-pipeline
```

Step 2 — Create a virtual environment

A virtual environment is an isolated Python installation. It keeps this project's dependencies separate from every other project on your machine so they never conflict.

```
python3 -m venv .venv
```

Step 3 — Activate the virtual environment

```
# Linux / macOS
source .venv/bin/activate

# Windows
.venv\Scripts\activate
```

When active, your terminal prompt shows `(.venv)`. Every `pip install` and `python` command now uses the isolated environment.

2.3 Install dependencies

Step 4 — Install PyTorch

PyTorch is the deep learning framework this pipeline is built on. Install it separately first because the wheel (the binary package) differs by CUDA version.

```
# CPU only (for development, CI, or if you have no GPU)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

# GPU with CUDA (auto-detects your CUDA version)
pip install torch torchvision
```

Step 5 — Install the pipeline package and dev tools

```
pip install -e ".[dev]"
```

WHAT DOES THIS COMMAND DO?

`pip install -e` installs the package in "editable" mode — changes you make to the source code in `src/` take effect immediately without reinstalling. The `".[dev]"` part means "install this package plus the extras listed under `[dev]` in `pyproject.toml`", which includes `pytest`, `black`, `ruff`, `mypy`, `isort`, and `pre-commit`.

Step 6 — Optional extras

```
# For preparing real Sentinel-1/2 data (requires rasterio)
pip install -e ".[geo]"

# For FID / Inception Score evaluation
pip install -e ".[eval]"
```

Step 7 — Install pre-commit hooks

Pre-commit hooks are scripts that run automatically every time you do `git commit`. They check your code quality before it enters the repository.

```
pre-commit install
```

Step 8 — Download VGG16 weights

The pipeline uses a pre-trained VGG16 network (trained on ImageNet) as a perceptual loss function. It cannot be included in the git repository because it is 528 MB. Download it once:

```
make download-weights
```

This saves the file to `weights/vgg16-397923af.pth` and it is reused for all future training runs.

2.4 Verify the installation

Run these three commands. All should pass with no errors:

```
make lint          # checks code style – should say "All checks passed"
make typecheck     # checks types – should say "Success: no issues found"
pytest            # runs 68 tests – should say "68 passed"
```

If all three pass, your environment is correctly set up.

Quick smoke test (verify training works without real data):

```
python scripts/make_dummy_data.py # creates 50 synthetic image pairs
python scripts/train_pix2pix.py \
    training.epochs=2 training.num_workers=0 training.lambda_vgg=0.0
```

You should see loss values printed to the console. If no errors appear, the full training loop is working.

Project Structure — What Every File Does

gan-pipeline/	
├─ src/gan_pipeline/	← The Python package (importable after <code>pip install -e ".[dev]"</code>)
│ └─ data/	
│ │ └─ dataset.py	← ImageFolder dataset for DCGAN (CelebA etc.)
│ │ └─ paired_dataset.py	← SAR/EO dataset classes + dataloader factory
│ │ └─ transforms.py	← torchvision preprocessing transforms
│ └─ sentinel_utils.py	← SAR/EO preprocessing math (pure numpy, no rasterio)
│ └─ models/	
│ │ └─ base.py	← Abstract base classes all models inherit from
│ │ └─ unet.py	← U-Net Generator (main model, 54M params)
│ │ └─ patchgan.py	← PatchGAN Discriminator (70×70 receptive field)
│ │ └─ multiscale_disc.py	← Multi-scale wrapper around PatchGAN
│ │ └─ dcgan.py	← DCGAN Generator + Discriminator (unconditional)
│ └─ losses.py	← All loss functions (hinge, BCE, WGAN, VGG, FM)
│ └─ training/	
│ │ └─ pix2pix_trainer.py	← Main training loop for SAR→EO translation
│ └─ trainer.py	← Training loop for unconditional DCGAN
│ └─ evaluation/	
│ └─ metrics.py	← FID and Inception Score via torch-fidelity
│ └─ inference/	
│ └─ generate.py	← Load checkpoint and generate images
│ └─ utils/	
│ │ └─ checkpointing.py	← Save/load model checkpoints
│ └─ logging.py	← Loguru logger setup
├─ configs/	← Hydra YAML configuration tree
│ └─ config.yaml	← Root: picks which model/training/data configs to load
│ └─ model/pix2pix.yaml	← U-Net + PatchGAN architecture params
│ └─ model/dcgan.yaml	← DCGAN architecture params
│ └─ training/pix2pix.yaml	← Learning rates, loss weights, epochs
│ └─ training/default.yaml	← DCGAN training params
│ └─ data/sar_eo.yaml	← Dataset paths, image size, channels
│ └─ data/celeba.yaml	← CelebA dataset config (DCGAN example)
├─ scripts/	← Executable entry points (run these directly)
│ └─ train_pix2pix.py	← START HERE for SAR→EO training
│ └─ train.py	← Entry point for unconditional DCGAN

├─ prepare_data.py	← Convert Sentinel GeoTIFFs → training PNGs
├─ evaluate.py	← Run FID/IS on a trained checkpoint
├─ generate.py	← Generate images from a checkpoint
├─ download_vgg_weights.py	← Download VGG16 weights to weights/
└─ make_dummy_data.py	← Generate synthetic SAR/E0 pairs for testing
├─ tests/	← 68 automated tests (run with pytest)
│ └─ conftest.py	← Shared fixtures (device, config objects)
│ └─ test_pix2pix.py	← 29 tests for the main pipeline
│ └─ test_sentinel_utils.py	← 19 tests for preprocessing
│ └─ test_models.py	← 8 tests for DCGAN models and losses
│ └─ test_data.py	← 4 tests for transforms
│ └─ test_training.py	← 2 tests for checkpoint and DCGAN trainer
├─ docker/	
│ └─ Dockerfile	← PyTorch CUDA runtime image
│ └─ docker-compose.yml	← Services: train, mlflow, generate
├─ .claude/skills/	← Claude Code AI workflow skills
│ └─ smoke-test.md	
│ └─ train-run.md	
│ └─ eval-run.md	
│ └─ new-model.md	
│ └─ prep-data.md	
│ └─ commit.md	
├─ weights/	← VGG16 weights live here (gitignored)
│ └─ .gitkeep	← Keeps the empty directory in git
├─ .github/workflows/	
│ └─ pre-commit.yml	← CI: lint + typecheck + tests
│ └─ pytest.yml	← CI: full test suite
├─ .pre-commit-config.yaml	← Defines the 5 pre-commit hooks
├─ pyproject.toml	← Package metadata + all tool configuration
├─ Makefile	← Developer shortcuts (make train, make lint, etc.)
├─ CLAUDE.md	← Instructions loaded into every Claude Code session
└─ .gitignore	← Excludes data/, outputs/, mlflow.db, *.pth, etc.

04

CHAPTER FOUR Configuration System (Hydra)

All hyperparameters — learning rates, batch size, number of layers, loss weights — are stored in YAML files, not hardcoded in Python. The tool that manages this is **Hydra**.

4.1 Why use Hydra instead of hardcoding?

Hardcoding values like `lr = 0.0002` inside Python files means you have to edit source code every time you want to try different settings. Hydra externalises all of these into config files, which means:

- You can change any setting from the command line without touching Python files
- Every experiment is reproducible — the config used is saved automatically
- You can swap entire config groups (e.g. switch from pix2pix to DCGAN) in one flag

4.2 How the config tree is assembled

The root file `configs/config.yaml` declares which sub-configs to load:

```
defaults:
  - model: pix2pix          # loads configs/model/pix2pix.yaml
  - training: pix2pix       # loads configs/training/pix2pix.yaml
  - data: sar_eo            # loads configs/data/sar_eo.yaml
  - _self_

experiment_name: sar_eo_pix2pix
seed: 42
output_dir: outputs/${experiment_name}
```

At runtime, Hydra merges all these files into one big config object that the training script receives.

4.3 Key parameters in training/pix2pix.yaml

Parameter	Default	What it controls
<code>epochs</code>	200	How many full passes through the dataset
<code>batch_size</code>	1	Images per training step (pix2pix standard)

<code>lr_generator</code>	0.0002	Generator learning rate
<code>lr_discriminator</code>	0.0002	Discriminator learning rate
<code>beta1</code>	0.5	Adam optimiser momentum (lower = less momentum = more stable GAN)
<code>loss_type</code>	hinge	Adversarial loss: <code>hinge</code> , <code>bce</code> , <code>wasserstein</code>
<code>lambda_l1</code>	100.0	Weight of pixel-level L1 loss
<code>lambda_vgg</code>	10.0	Weight of VGG perceptual loss
<code>lambda_fm</code>	10.0	Weight of feature matching loss
<code>vgg_weights_path</code>	weights/ vgg16-397923af.pth	Path to VGG16 weights for offline use

4.4 Overriding from the command line

```
# Change the loss type
python scripts/train_pix2pix.py training.loss_type=bce

# Change multiple values at once
python scripts/train_pix2pix.py \
  experiment_name=my_experiment \
  training.epochs=50 \
  training.lambda_vgg=0.0 \
  model.discriminator.n_scales=2

# Switch to DCGAN entirely
python scripts/train.py model=dcgan training=default data=celeba
```

TIP

Hydra automatically saves the merged config to `outputs/<experiment_name>/hydra/config.yaml` so you always know exactly what settings were used for each run.

Data Pipeline — From Raw Satellite to Training Tensors

5.1 Raw Sentinel data

Raw data from ESA comes as GeoTIFF files — multi-channel raster images with geographic metadata. Before training, these must be converted to simple PNGs that PyTorch can load.

The conversion script is `scripts/prepare_data.py`. It supports two modes:

Mode	Input	Use when...
<code>sen12ms</code>	Pre-cropped 256×256 GeoTIFF patches from the SEN12MS benchmark dataset	You have SEN12MS downloaded — fastest to get started
<code>scenes</code>	Full Sentinel-1 and Sentinel-2 scene GeoTIFFs	You have your own raw satellite data and want to chip it into patches

Example command for SEN12MS:

```
python scripts/prepare_data.py \  
  --mode sen12ms \  
  --s1-dir /data/SEN12MS/s1 \  
  --s2-dir /data/SEN12MS/s2 \  
  --output-dir data/sar_eo \  
  --sar-already-db \      # SEN12MS stores SAR already in dB scale  
  --sar-channels 1 \      # 1 = VV only (grayscale), 3 = VV+VH+VV (pseudo-RGB)  
  --val-split 0.1 \  
  --test-split 0.1
```

5.2 Preprocessing (sentinel_utils.py)

The file `src/gan_pipeline/data/sentinel_utils.py` contains pure-numpy functions that convert raw sensor values to uint8 images. No rasterio dependency — these can be imported and used anywhere.

SAR preprocessing pipeline

```
Raw Sentinel-1 GRD backscatter (float32, linear power scale)  
|  
▼ 10 · log10(max(x, 1e-10))    ← convert to dB. The 1e-10 floor
```

```

|                                     prevents log(0) = -∞
▼
dB values (typically -25 dB to 0 dB for land)
|
▼ clip to [-25, 0] dB                ← values below -25 are sensor noise;
                                     values above 0 are very bright targets
|
▼
(x - min_db) / (max_db - min_db)    ← normalise to [0, 1]
|
▼ × 255 → round → uint8             ← ready to save as PNG

```

EO preprocessing pipeline

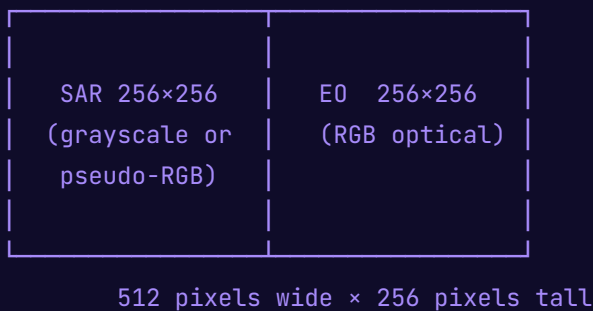
```

Raw Sentinel-2 L2A reflectance (uint16, values = reflectance × 10000)
|
▼ ÷ 10000                          ← convert to physical reflectance [0, ~1]
|
▼ clip to [0, 0.3]                 ← most land surfaces < 0.3 reflectance;
                                     snow/clouds saturate – acceptable
|
▼ ÷ 0.3                            ← normalise to [0, 1]
|
▼ × 255 → uint8                    ← ready to save as PNG

```

Output format

Both SAR and EO patches are assembled into a single **side-by-side PNG**: SAR on the left half, EO on the right half. For 256×256 target images this is a 512×256 file.



The dataset loader splits this back at load time. Storing them together makes shuffling and splitting the dataset simpler.

5.3 Dataset classes (paired_dataset.py)

Two dataset classes handle loading:

SideBySidePairedDataset

Opens each PNG, splits it at the midpoint horizontally, returns a dict with SAR and EO tensors.

```
dataset = SideBySidePairedDataset(
    root="data/sar_eo",
    split="train",          # looks inside data/sar_eo/train/
    image_size=256,
    sar_channels=1,         # how many channels to use for SAR
    eo_channels=3,          # always 3 for RGB optical
    augment=True,           # False for val and test
)
# Each item: {"sar": Tensor(1, 256, 256), "eo": Tensor(3, 256, 256)}
# Values normalised to [-1, 1]
```

SeparateDirPairedDataset

Alternative layout: SAR in `trainA/`, EO in `trainB/`. Filenames must match. Useful if you already have your data in CycleGAN-style directories.

5.4 Data augmentation

Augmentation increases the effective size of the dataset by applying random transformations at load time. The key constraint: **the same transformation must be applied to both SAR and EO**. If you flip one, you must flip the other — otherwise the pair is no longer aligned.

This is achieved using `torchvision.transforms.functional` with shared random parameters:

Step	What happens	Why
1. Resize	Both images scaled to 286×286	Makes the random crop always possible
2. Random crop	Same random 256×256 region cut from both	Spatial jitter without misalignment
3. Random horizontal flip	Same flip/no-flip decision applied to both	Doubles effective dataset size
4. Normalise	uint8 [0,255] → float [-1,1]	Neural networks train better on zero-centred inputs

Augmentation is disabled for validation and test splits to ensure consistent evaluation.

5.5 Weighted Random Sampling

The SEN12MS dataset is not class-balanced. Forest and cropland patches make up ~65% of the data. Water and snow patches are under 2% each. If you train on this distribution as-is, the model learns forest and crops very well and performs poorly on rare classes.

Weighted Random Sampling solves this by upsampling rare classes. Each sample is assigned a weight equal to the inverse frequency of its class. The DataLoader then samples

according to these weights with replacement, so every class is seen approximately equally per epoch.

```
from collections import Counter
from torch.utils.data import WeightedRandomSampler

# land-cover class label for each training image
train_classes = [12, 1, 17, 12, 1, ...] # loaded from manifest.csv

# weight = total_samples / count_of_this_class
class_counts = Counter(train_classes)
total = sum(class_counts.values())
class_weight = {cls: total / cnt for cls, cnt in class_counts.items()}

# one weight per sample
sample_weights = [class_weight[cls] for cls in train_classes]

sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True, # same sample can appear multiple times per epoch
)

# Pass sampler instead of shuffle=True
loader = DataLoader(dataset, batch_size=1, sampler=sampler)
```

IMPORTANT

When using a sampler, you must set `shuffle=False` in the `DataLoader` (or not pass it at all). Passing both `sampler` and `shuffle=True` raises an error.

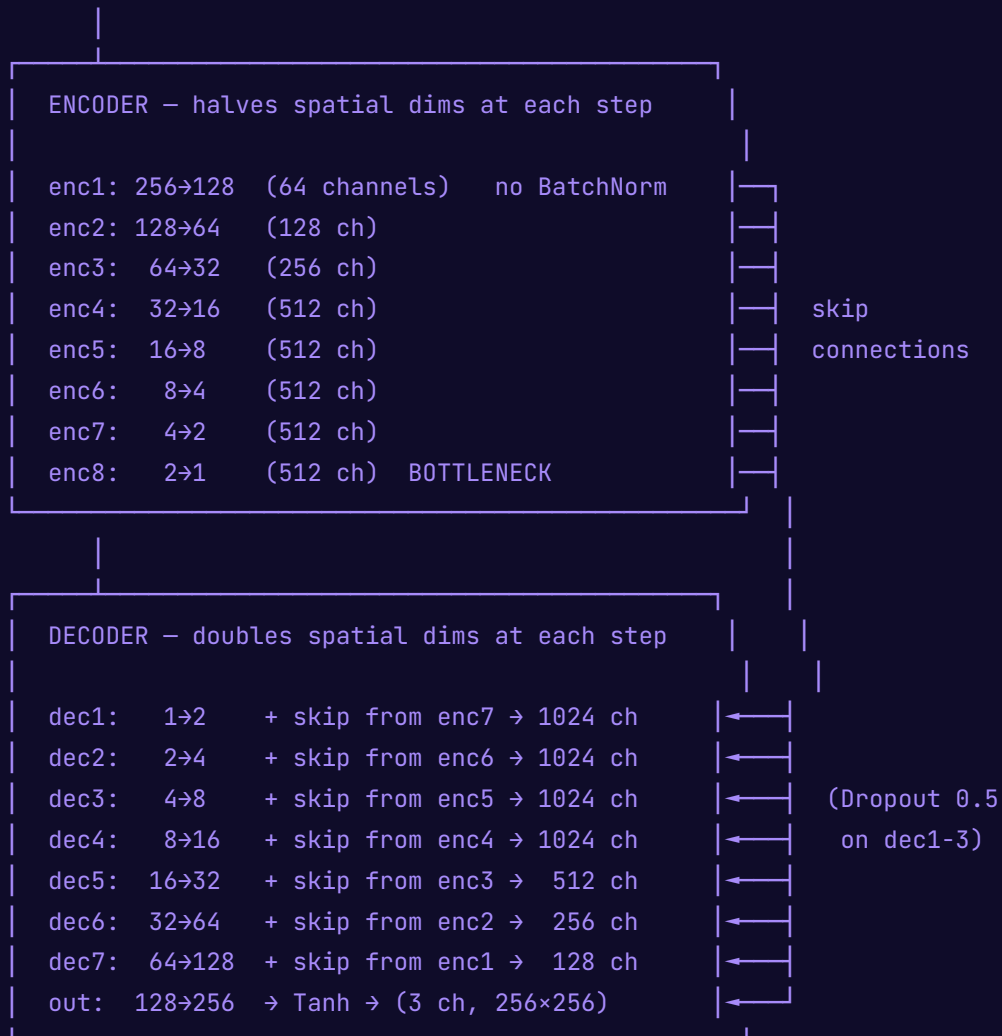
6.1 U-Net Generator (models/unet.py)

Class: `UNetGenerator(in_channels, out_channels, base_features=64)`

Parameters: ~54 million

The generator is an 8-level U-Net. The "U" shape comes from an encoder that progressively shrinks the image, followed by a decoder that progressively restores it. Skip connections pass information directly from each encoder level to its matching decoder level.

SAR input (1 ch, 256×256)



Fake E0 output (3 ch, 256×256, values in [-1, 1])

Why skip connections? The bottleneck (1×1 spatial) forces the network to compress all information into a tiny representation. Low-level details like edges and textures would be lost.

Skip connections bypass the bottleneck by sending early-layer features directly to later decoder layers, preserving fine spatial information.

CRITICAL CONSTRAINT — NEVER CHANGE

All ReLU activations in the decoder use `ReLU(inplace=False)`. If you change them to `inplace=True`, PyTorch will raise a `RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation`. This is because the skip connections share tensor memory with the encoder, and inplace modification corrupts the values needed for the backward pass.

6.2 PatchGAN Discriminator (models/patchgan.py)

Class: `PatchGANDiscriminator(sar_channels, eo_channels, base_features=64, spectral_norm=False)`

Instead of outputting a single "real or fake" score for the whole image, PatchGAN outputs a grid of scores — one score per patch of the image. Each score represents "is this patch real or fake?"

```
Input: concat([SAR, EO or fake_EO]) = (B, 4, 256, 256)
      |
      L1: Conv(4→64, 4×4, stride=2) LeakyReLU no BN → (B, 64, 128, 128)
      L2: Conv(64→128, 4×4, stride=2) BN LeakyReLU → (B, 128, 64, 64)
      L3: Conv(128→256, 4×4, stride=2) BN LeakyReLU → (B, 256, 32, 32)
      L4: Conv(256→512, 4×4, stride=1) BN LeakyReLU → (B, 512, 31, 31)
      L5: Conv(512→1, 4×4, stride=1) → (B, 1, 30, 30)
      |
Output: (B, 1, 30, 30) - 900 independent patch scores
Each score covers a 70×70 pixel receptive field in the original image
```

Why PatchGAN? A full-image discriminator outputs one scalar and gives the generator a weak, diffuse signal. PatchGAN gives 900 independent feedback signals covering every region of the image — the generator gets penalised for any local patch that looks fake, producing sharper textures.

Spectral Normalisation: When enabled (`spectral_norm=True`), `nn.utils.spectral_norm` is applied to every Conv2d layer. This constrains the Lipschitz constant of the discriminator, preventing it from becoming too powerful and destabilising training. Applied after weight initialisation so the $N(0, 0.02)$ initialisation is preserved.

6.3 Multi-Scale Discriminator (models/multiscale_disc.py)

Class: `MultiScaleDiscriminator(sar_channels, eo_channels, base_features=64, n_scales=3, spectral_norm=False)`

Runs N independent PatchGAN discriminators on progressively downsampled versions of the input. Each discriminator has its own weights and evaluates a different aspect of the image quality.

```
Input pair (B, 4, 256, 256)
  |
  ├──> D0 at full res (256×256) → 30×30 patch map    ← fine texture, sharp edges
  |
  | AvgPool2d(3, stride=2)    ← smooth downsampling, no aliasing
  |
  ├──> D1 at half res (128×128) → 14×14 patch map    ← mid-level structure
  |
  | AvgPool2d(3, stride=2)
  |
  └──> D2 at quarter res (64×64) → 6×6 patch map    ← global layout, composition

All three patch maps are collected in a list → losses are averaged across scales
```

Why multi-scale? A single fine-scale discriminator can miss global inconsistencies — the generator might produce a locally-convincing texture that doesn't make sense globally (e.g. a forest that looks fine patch by patch but has inconsistent tree patterns across the image). The coarse-scale discriminators catch these global failures.

IMPORTANT IMPLEMENTATION DETAIL

`MultiScaleDiscriminator.forward()` returns a `List[Tensor]`, not a single tensor. This is why there are separate `multiscale_discriminator_loss()` and `multiscale_generator_loss()` functions in `losses.py` — they average the loss across all scales.

6.4 DCGAN Models (`models/dcgan.py`)

Used only in the unconditional pipeline (`scripts/train.py`). Takes random noise, generates an image — no conditioning on SAR input. Useful for training on datasets like CelebA to verify the architecture and training loop work before tackling the harder SAR→EO task.

Loss Functions — What the Model Optimises and Why

The model is trained by minimising a loss function — a number that measures how bad the current output is. Multiple loss terms are combined, each measuring a different quality dimension.

7.1 Adversarial Loss

The adversarial loss is the core GAN signal — it measures how well the Generator is fooling the Discriminator.

Hinge Loss (default — recommended)

```
L_D = mean(relu(1 - real_logits)) + mean(relu(1 + fake_logits))
L_G = -mean(fake_logits)
```

Derived from the SVM margin objective. Real samples are rewarded only if their score is above +1; fake samples are penalised only if their score is above -1. This creates a stable "margin" that prevents gradient explosion when the discriminator is very confident.

BCE Loss

```
L_D = 0.5 × (BCE(real_logits, 1) + BCE(fake_logits, 0))
L_G = BCE(fake_logits, 1)
```

Standard binary cross-entropy. Simple and well-understood, but can suffer from vanishing gradients when the discriminator becomes very confident early in training.

Wasserstein Loss

```
L_D = mean(fake_logits) - mean(real_logits)
L_G = -mean(fake_logits)
```

Measures the "Earth Mover's Distance" between real and generated distributions. Requires gradient penalty (`training.gradient_penalty: true`) to satisfy the Lipschitz constraint — without it the discriminator can diverge.

7.2 L1 Pixel Loss

$$L_{L1} = \text{mean}(|\text{fake_E0} - \text{real_E0}|)$$

Measures the average absolute pixel difference between the generated and real image.

Weight: $\lambda_{L1} = 100$ — much higher than the adversarial weight because pixel accuracy is a primary goal.

Why L1 and not L2? L2 (mean squared error) penalises large errors more heavily, which causes the model to produce blurry outputs (the "safe" average of all possible outputs). L1 allows sharper predictions.

7.3 VGG Perceptual Loss

Pixel loss measures low-level accuracy but doesn't capture perceptual quality. Two images can look visually very different but have similar pixel-level L1 distances (e.g. a shifted version of the same image).

The perceptual loss extracts feature maps from a pre-trained VGG16 network at four intermediate layers and measures their L1 distance:

$$L_{VGG} = \sum L1(VGG_i(\text{fake_E0}), VGG_i(\text{real_E0})) \quad \text{for } i \text{ in } \{\text{relu1_2}, \text{relu2_2}, \text{relu3_3}, \text{relu4_3}\}$$

Each VGG layer captures increasingly abstract features:

VGG Layer	Captures	What it encourages
relu1_2	Edges, colours	Sharp edges, correct colour tones
relu2_2	Textures	Realistic surface textures
relu3_3	Patterns	Mid-level structures (roads, field boundaries)
relu4_3	Semantics	High-level content (building vs vegetation)

The VGG network is frozen (weights don't change during training) — it is purely a feature extractor. Weight: $\lambda_{VGG} = 10$.

7.4 Feature Matching Loss

Introduced in Pix2PixHD. Instead of only using the Discriminator's final output, this loss also uses the Discriminator's intermediate features:

```
L_FM = mean over (scale s, layer l) of: L1(D_s_l(fake_pair),  
D_s_l(real_pair).detach())
```

This pushes the Generator to produce images that activate the Discriminator's internal features in the same way that real images do — a much richer training signal than just the final real/fake score. Weight: **$\lambda_{FM} = 10$** .

Real features are `.detach()`-ed so gradients only flow through the fake path to update the Generator.

7.5 Full Generator Loss

```
L_G_total = L_G_adv          ← fool the discriminator  
            + 100.0 × L1(fake_E0, real_E0)    ← stay close to real pixels  
            + 10.0 × L_VGG(fake_E0, real_E0) ← match perceptual features  
            + 10.0 × L_FM          ← match discriminator internals
```

Setting any lambda to 0.0 disables that loss term entirely without changing any code:

```
python scripts/train_pix2pix.py training.lambda_vgg=0.0 training.lambda_fm=0.0
```

Trainers — The Training Loop Explained

8.1 What a training loop does

Training a neural network is a loop: show the model a batch of data, measure how wrong it is (loss), compute how to improve (gradients via backpropagation), and update the weights (optimiser step). Repeat millions of times.

GANs are more complex because there are two networks (G and D) that must be updated alternately.

8.2 Pix2PixTrainer — one training step

File: `src/gan_pipeline/training/pix2pix_trainer.py`

For each batch `{"sar": ..., "eo": ...}`:

1

Generate a fake EO image

```
fake_eo = generator(sar).detach()
```

The `.detach()` prevents gradients from flowing back into the Generator during the Discriminator update — we only want to update D here.

2

Update the Discriminator

Forward the real pair `[sar, real_eo]` and the fake pair `[sar, fake_eo]` through D. Compute D loss. Backpropagate. Optimiser step.

3

Generate again (without detach)

```
fake_eo = generator(sar)
```

This time we keep the computational graph so gradients can flow back into G.

4

Update the Generator

Forward the fake pair through D. Compute G total loss (adversarial + L1 + VGG + FM). Backpropagate through both D and G. Optimiser step on G only.

5

Log and checkpoint

Log losses to console and MLflow. Save checkpoint and sample images at configured intervals.

8.3 Checkpoint format

Checkpoints are saved as `.pt` files in `outputs/<experiment_name>/checkpoints/`:

```
{
  "epoch": int,
  "generator": OrderedDict,      # generator.state_dict()
  "discriminator": OrderedDict,  # discriminator.state_dict()
  "opt_g": dict,                 # generator optimiser state
  "opt_d": dict,                 # discriminator optimiser state
  "metrics": {
    "d_loss", "g_adv", "g_l1", "g_vgg", "g_fm"
  }
}
```

8.4 Resuming from a checkpoint

```
python scripts/train_pix2pix.py \
  resume=outputs/sar_eo_pix2pix/checkpoints/epoch_0050.pt
```

The trainer loads all weights and optimiser states and continues from the saved epoch.

8.5 What to watch during training

Signal	Healthy	Problem
<code>d_loss</code>	0.3–1.5, oscillating	Collapses to ~0 and stays there (D overpowers G — mode collapse risk)
<code>g_adv</code>	0.5–2.0, slowly rising	Very high (>5) and rising (G cannot fool D at all)
<code>g_l1</code>	Slowly decreasing over epochs	Stuck or increasing (G not learning pixel fidelity)
<code>g_vgg</code>	Slowly decreasing	NaN (VGG weights issue or numerical instability)
Any loss	Finite	NaN — check learning rate (too high) or data normalisation

Makefile — Developer Shortcuts

The `Makefile` provides short memorable commands for all common tasks. Instead of remembering long command strings, you type `make <target>`.

Command	What it runs	When to use it
<code>make venv</code>	<code>python3 -m venv .venv + pip upgrade</code>	First time setup
<code>make install</code>	<code>pip install -e ".[dev]"</code>	After pulling new changes that added dependencies
<code>make lint</code>	<code>ruff + black --check + isort --check</code>	Before committing, to check code style
<code>make format</code>	<code>black + isort + ruff --fix (in-place)</code>	To auto-fix style issues before committing
<code>make typecheck</code>	<code>mypy src/</code>	To check for type errors
<code>make test</code>	<code>pytest</code>	Run the full test suite
<code>make download-weights</code>	<code>python scripts/download_vgg_weights.py</code>	One-time after cloning
<code>make prepare-data</code>	<code>python scripts/prepare_data.py ...</code>	When you have raw Sentinel data to prepare
<code>make train</code>	<code>python scripts/train_pix2pix.py</code>	Start a training run with default settings
<code>make train-dcgan</code>	<code>python scripts/train.py model=dcgan ...</code>	Train the unconditional DCGAN
<code>make mlflow</code>	<code>mlflow ui --backend-store-uri sqlite:///mlflow.db</code>	View training metrics in browser
<code>make docker-build</code>	<code>docker compose build</code>	Build Docker images
<code>make docker-train</code>	<code>docker compose up train</code>	Train inside Docker
<code>make clean</code>		Clean up build artefacts

Remove

`__pycache__`, `.pytest_cache`, etc.

`make clean-venv`

`rm -rf .venv`

Remove the virtual environment

`make clean-all`

clean + clean-venv

Full reset

Overridable variables:

```
PYTHON=python3.11 make venv
VENV_DIR=.myenv make venv
```

```
# use Python 3.11 instead of python3
# create venv at a custom path
```

Pre-commit Hooks — Automated Quality Gates

A **pre-commit hook** is a script that runs automatically every time you type `git commit` — before the commit is created. If the script fails, the commit is blocked.

This guarantees that no broken, unformatted, or type-error code can ever enter the repository — even accidentally.

10.1 Setup (one time)

```
pre-commit install
```

This registers the hooks in your local `.git/hooks/` directory. You only run this once per machine after cloning.

10.2 The five hooks

Hook	Tool	What it checks	How to fix failures
ruff lint	ruff	Unused imports, bad style (PEP8), deprecated syntax	<code>ruff check --fix src tests scripts</code>
black format	black	Code formatting (line length, spacing, quotes)	<code>black src tests scripts</code>
isort imports	isort	Import statement ordering (stdlib → third-party → local)	<code>isort src tests scripts</code>
mypy typecheck	mypy	Type annotations are correct (strict mode)	Fix the type error in the reported file
pytest	pytest	All 68 tests pass	Fix the failing test or the code it tests

10.3 Typical workflow

```
# Make your changes
# ...edit files...
```

```
# Check formatting first (fastest feedback)
make format
make lint

# Stage your files
git add src/gan_pipeline/models/myfile.py

# Commit – all 5 hooks run automatically
git commit -m "Add ResNet generator variant"

# If a hook fails, the commit is blocked with an error message
# Fix the issue, re-stage, and try again
```

10.4 Running hooks manually

```
# Run all hooks on all files (useful before opening a PR)
pre-commit run --all-files

# Run just one hook
pre-commit run black --all-files
```

10.5 Emergency bypass

```
git commit --no-verify -m "wip: in progress, do not merge"
```

WARNING

Only use `--no-verify` for work-in-progress commits you will fix before merging. Never use it on a commit that will be pushed to the main branch.

10.6 Tool configuration

All tools share the same settings from `pyproject.toml`: line length 100, Python 3.10 target, mypy strict mode.

CI/CD Workflows (GitHub Actions)

CI/CD stands for Continuous Integration / Continuous Deployment. In this project it means: every time code is pushed to GitHub, automated checks run in the cloud to verify nothing is broken.

Two workflow files live in `.github/workflows/` :

11.1 pre-commit.yml

Runs all five pre-commit hooks (`pre-commit run --all-files`) in a fresh Ubuntu environment. This catches any case where someone bypassed pre-commit locally using `--no-verify` .

```
on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
  pre-commit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10"
          cache: pip
      - run: pip install -e ".[dev]"
      - run: pre-commit run --all-files
```

11.2 pytest.yml

Runs the full test suite independently. Ensures tests pass even if pre-commit is configured differently on someone's machine.

11.3 Status badges

The README displays live badges for each workflow. A green badge means the last push passed all checks. A red badge means something is broken and needs attention.

WORKFLOW FOR CONTRIBUTING CHANGES

Make your changes → run `pre-commit run --all-files` locally → push → confirm both CI badges turn green → your change is safe to merge.

MLflow — Experiment Tracking

MLflow is a platform for tracking machine learning experiments. Every training run automatically records its configuration and metrics so you can compare experiments and reproduce results.

12.1 What is logged automatically

Type	What	When
Parameters	loss_type, all λ values, n_scales, learning rates, batch size, model name	Once at the start of each run
Metrics	d_loss, g_adv, g_l1, g_vgg, g_fm	Once per epoch

12.2 Where data is stored

MLflow 3.x stores experiment data in a local SQLite database: `mlflow.db` in the project root. This file is excluded from git by `.gitignore`.

12.3 Viewing the UI

```
make mlflow
```

Opens at `http://localhost:5000`. The UI shows:

- **Experiments** — one per unique `experiment_name` value
- **Runs** — one row per training run with all parameters and final metrics
- **Metric charts** — plot any metric over epochs, overlay multiple runs
- **Comparison** — select two or more runs to compare side by side

12.4 Naming experiments for comparison

```
# Run A — hinge loss, 3 scales
python scripts/train_pix2pix.py experiment_name=hinge_3scale

# Run B — BCE loss, 1 scale
```

```
python scripts/train_pix2pix.py \  
    experiment_name=bce_1scale \  
    training.loss_type=bce \  
    model.discriminator.n_scales=1  
  
# View both in MLflow UI side by side  
make mlflow
```

12.5 Stop MLflow

```
pkill -f "mlflow ui"
```


Docker — Reproducible Environments

Docker packages the entire environment — Python version, CUDA drivers, all dependencies — into a container that runs identically on any machine. This eliminates "works on my machine" problems.

13.1 Services

Service	What it does	When to use
<code>train</code>	Runs <code>scripts/train_pix2pix.py</code> with GPU access	Production training on a GPU server
<code>mlflow</code>	MLflow tracking server at localhost:5000	When you want MLflow accessible over the network
<code>generate</code>	Runs <code>scripts/generate.py</code> to produce images from a checkpoint	Inference after training

13.2 Build and run

```
# Build all images (downloads PyTorch CUDA base — takes a while first time)
make docker-build

# Train
make docker-train

# Generate images from a trained checkpoint
docker compose -f docker/docker-compose.yml \
  --profile generate run generate \
  checkpoint=outputs/sar_eo_pix2pix/checkpoints/epoch_0199.pt
```

13.3 Volumes

The Docker setup mounts your local directories into the container so data and outputs persist after the container stops:

- `data/` → training data is read from your host machine
- `outputs/` → checkpoints and sample images are saved to your host machine

- `weights/` → VGG16 weights are read from your host machine
- `mlruns/` → MLflow data persists across container restarts

13.4 Environment variables

```
cp .env.example .env
# Edit .env:
MLFLOW_TRACKING_URI=http://localhost:5000
CUDA_VISIBLE_DEVICES=0    # which GPU to use
```

Claude Code Skills — AI Workflow Automation

Claude Code is an AI coding assistant integrated with this project. **Skills** are reusable instruction modules — you invoke them with a `/` command and Claude follows the steps defined in the skill file. They are lazy-loaded, meaning they only use context when actually invoked.

Skill files live in `.claude/skills/`. The `CLAUDE.md` file at the project root is loaded automatically every session and gives Claude context about the project.

14.1 Available skills

Skill	File	What it does
<code>/smoke-test</code>	<code>smoke-test.md</code>	Generates 50 dummy image pairs and runs 3 training epochs with all losses to verify the full pipeline end-to-end
<code>/train-run</code>	<code>train-run.md</code>	Interactively configures a named training run — prompts for experiment name, loss type, loss weights, epochs — then builds and executes the command
<code>/eval-run</code>	<code>eval-run.md</code>	Evaluates a checkpoint with FID and Inception Score — prompts for paths, checks dependencies, runs evaluation
<code>/new-model</code>	<code>new-model.md</code>	Step-by-step checklist for adding a new architecture: create file, export, add config, wire into trainer, write tests, run smoke test
<code>/prep-data</code>	<code>prep-data.md</code>	Guides through preparing Sentinel-1/2 data — prompts for mode, paths, channels, splits — then runs <code>prepare_data.py</code>
<code>/commit</code>	<code>commit.md</code>	Stages and commits changes following project conventions — checks git diff, drafts message, runs hooks, never adds Co-Authored-By

14.2 How to invoke a skill

In the Claude Code chat, type the skill name:

Claude loads the skill file and follows the steps defined in it.

14.3 CLAUDE.md

The file `CLAUDE.md` at the project root is automatically loaded into every Claude Code session. It contains:

- All make commands and their purpose
- The full architecture overview
- Pre-commit hooks and CI workflows
- Critical constraints (like the ReLU inplace rule)
- The skills table
- Git commit conventions (no Co-Authored-By)

This means Claude understands the project from the first message of every session without you having to re-explain it.

Testing — How the Codebase Verifies Itself

The project has 68 automated tests that verify every major component works correctly. Tests are in the `tests/` directory and run with `pytest`.

15.1 Running tests

```
pytest                # run all 68 tests
pytest -v             # verbose — shows each test name
pytest tests/test_pix2pix.py  # one file
pytest -k "vgg or feature"  # tests whose name contains these words
pytest --cov=gan_pipeline --cov-report=term-missing  # with coverage report
```

15.2 What each test file covers

File	Tests	Coverage
test_pix2pix.py	29	U-Net output shapes, skip-connection gradients, PatchGAN patch map shapes, spectral norm, multi-scale outputs, all loss types, VGG loss (offline weights, various channel counts), feature matching loss, full train step, dataset augmentation
test_sentinel_utils.py	19	linear_to_db correctness and zero-safety, SAR/EO normalisation, make_sar_image channel configs, make_eo_image shape, is_valid_patch NaN/zero rejection, side-by-side assembly
test_models.py	8	DCGAN generator/discriminator output shapes, all three adversarial loss types, gradient penalty, sample() method
test_data.py	4	Transform output shape at 32/64/128 px, pixel range [-1, 1]
test_training.py	2	Checkpoint save/load round-trip, DCGAN trainer step with finite losses

15.3 Test conventions

- All tests run on CPU — no GPU required

- VGG perceptual loss tests use `lambda_vgg=0.0` to avoid requiring the 528 MB weights file
- Shared fixtures (device, config) are in `tests/conftest.py`
- Tests are part of the pre-commit hook — every commit must pass all 68 tests
- Tests verify behaviour, not implementation — if a test breaks after refactoring, something actually changed

End-to-End Workflow — First Training Run

This chapter walks through the complete workflow from a freshly cloned repository to a finished training run. Follow these steps in order.

1

Clone and create environment

```
git clone https://github.com/baudhya/gan-pipeline.git
cd gan-pipeline
python3 -m venv .venv
source .venv/bin/activate
```

2

Install dependencies

```
pip install torch torchvision # GPU version (CPU: add --index-url flag)
pip install -e ".[dev]"
```

3

Install pre-commit hooks

```
pre-commit install
```

4

Download VGG16 weights

```
make download-weights # ~528 MB, saves to weights/vgg16-397923af.pth
```

5

Verify installation

```
make lint && make typecheck && pytest
```

All should pass with no errors.

6

Prepare training data

Option A — Real Sentinel data (install geo extras first):

```

pip install -e ".[geo]"
python scripts/prepare_data.py \
  --mode sen12ms \
  --s1-dir /path/to/s1 \
  --s2-dir /path/to/s2 \
  --output-dir data/sar_eo \
  --sar-already-db

```

Option B — Dummy data for testing:

```
python scripts/make_dummy_data.py
```

7

Start training

```

# With all default settings (hinge loss, 3 scales, VGG + FM enabled)
python scripts/train_pix2pix.py experiment_name=my_first_run

# Or with custom settings
python scripts/train_pix2pix.py \
  experiment_name=hinge_novgg \
  training.lambda_vgg=0.0 \
  training.lambda_fm=0.0 \
  training.epochs=50

```

8

Monitor training

In a second terminal, launch MLflow:

```
make mlflow # open http://localhost:5000
```

Checkpoints are saved to `outputs/my_first_run/checkpoints/`

Sample images are saved to `outputs/my_first_run/samples/`

9

Evaluate a trained checkpoint

```

pip install -e ".[eval]"
python scripts/evaluate.py \
  checkpoint=outputs/my_first_run/checkpoints/epoch_0199.pt \
  real_dir=data/sar_eo/test \
  eval_samples=1000

```



```
python scripts/generate.py \  
  checkpoint=outputs/my_first_run/checkpoints/epoch_0199.pt \  
  gen_output_dir=outputs/generated
```

YOU ARE READY

You now understand every component of this pipeline — what it does, why it exists, and how to operate it. For day-to-day work, use the `make` shortcuts and Claude Code skills to move faster.

How to Build a Custom Dataset

This chapter explains how to implement a new dataset class from scratch — for example, if you have a different data format than side-by-side PNGs.

17.1 How PyTorch datasets work

A PyTorch `Dataset` is any class that implements two methods:

- `__len__(self)` — returns the total number of samples
- `__getitem__(self, idx)` — returns one sample given its index

The `DataLoader` wraps a dataset and handles batching, shuffling, and multi-process loading automatically.

17.2 Anatomy of SideBySidePairedDataset

```
class SideBySidePairedDataset(Dataset[dict[str, torch.Tensor]]):

    def __init__(
        self,
        root: str,
        split: str,          # "train" | "val" | "test"
        image_size: int,
        sar_channels: int,    # 1 = grayscale, 3 = pseudo-RGB
        eo_channels: int,
        augment: bool,
    ) → None:
        # Build a list of all PNG file paths in root/split/
        self.files = sorted(Path(root, split).glob("*.png"))
        self.image_size = image_size
        self.sar_channels = sar_channels
        self.augment = augment

    def __len__(self) → int:
        return len(self.files)    # total number of image pairs

    def __getitem__(self, idx: int) → dict[str, torch.Tensor]:
        img = Image.open(self.files[idx]).convert("RGB")

        # Split at the horizontal midpoint
        w, h = img.size
        sar_img = img.crop((0, 0, w // 2, h))    # left half = SAR
        eo_img = img.crop((w // 2, 0, w, h))    # right half = EO
```

```

# Apply the same augmentation to both
sar_img, eo_img = self._augment(sar_img, eo_img)

# Convert to tensors in [-1, 1]
return {
    "sar": self._to_tensor_normalized(sar_img, self.sar_channels),
    "eo": self._to_tensor_normalized(eo_img, 3),
}

```

17.3 Implementing your own dataset class

Suppose you have SAR and EO in HDF5 files instead of PNGs. Here is how to implement the dataset:

```

import h5py
import torch
from torch.utils.data import Dataset

class HDF5PairedDataset(Dataset[dict[str, torch.Tensor]]):

    def __init__(self, h5_path: str, split: str, image_size: int) → None:
        self.h5_path = h5_path
        self.split = split
        self.image_size = image_size
        # Count samples without loading data into memory
        with h5py.File(h5_path, "r") as f:
            self.length = len(f[self.split]["sar"])

    def __len__(self) → int:
        return self.length

    def __getitem__(self, idx: int) → dict[str, torch.Tensor]:
        # Open the file per call – h5py is thread-safe for reading
        with h5py.File(self.h5_path, "r") as f:
            sar = f[self.split]["sar"][idx] # numpy array
            eo = f[self.split]["eo"][idx]

        # Normalise to [-1, 1] and convert to tensor
        sar_t = torch.from_numpy(sar).float() / 127.5 - 1.0
        eo_t = torch.from_numpy(eo).float() / 127.5 - 1.0
        return {"sar": sar_t, "eo": eo_t}

# Use exactly like the built-in dataset
from torch.utils.data import DataLoader
dataset = HDF5PairedDataset("data/sentinel.h5", split="train", image_size=256)
loader = DataLoader(dataset, batch_size=4, shuffle=True, num_workers=4)

```

17.4 Registering a new dataset in the factory

The function `get_paired_dataloader` in `paired_dataset.py` reads `cfg.data.dataset_format` and instantiates the right class. To add your new format:

```
def get_paired_data_loader(cfg: DictConfig, split: str = "train") → DataLoader:
    fmt = cfg.data.dataset_format
    if fmt == "side_by_side":
        dataset = SideBySidePairedDataset(...)
    elif fmt == "separate_dirs":
        dataset = SeparateDirPairedDataset(...)
    elif fmt == "hdf5":
        dataset = HDF5PairedDataset(...) # ← add your format here
    else:
        raise ValueError(f"Unknown dataset_format: {fmt}")
    return DataLoader(dataset, ...)
```

Then set `dataset_format: hdf5` in `configs/data/sar_eo.yaml` or via CLI override.

17.5 Synchronized augmentation — the key pattern

When augmenting paired data, both images must receive identical spatial transforms. The pattern used in this codebase:

```
import torchvision.transforms.functional as TF
import random

def apply_paired_augmentation(
    sar: Image.Image, eo: Image.Image, image_size: int
) → tuple[Image.Image, Image.Image]:
    # Step 1 – resize both to slightly larger than target
    resize_to = int(image_size * 1.12)
    sar = TF.resize(sar, [resize_to, resize_to])
    eo = TF.resize(eo, [resize_to, resize_to])

    # Step 2 – compute ONE random crop, apply to BOTH
    i, j, h, w = transforms.RandomCrop.get_params(
        sar, output_size=(image_size, image_size)
    )
    sar = TF.crop(sar, i, j, h, w)
    eo = TF.crop(eo, i, j, h, w)

    # Step 3 – ONE flip decision, applied to BOTH
    if random.random() > 0.5:
        sar = TF.hflip(sar)
        eo = TF.hflip(eo)

    return sar, eo
```

How to Build a Custom Model

This chapter walks through building a new generator from scratch, registering it in the config system, and integrating it with the training pipeline.

18.1 The base class contract

Every model inherits from `BaseGenerator` or `BaseDiscriminator` in `models/base.py`. The contract:

```
from abc import ABC, abstractmethod
import torch
import torch.nn as nn

class BaseGenerator(nn.Module, ABC):

    @abstractmethod
    def forward(self, x: torch.Tensor) → torch.Tensor:
        ... # subclass must implement this

    def sample(self, n: int, device: torch.device) → torch.Tensor:
        # default: generate n samples from random noise
        z = torch.randn(n, self.latent_dim, device=device)
        return self(z) # calls forward(z)
```

18.2 Building a ResNet Generator step by step

ResNet generators (used in CycleGAN) use residual blocks instead of an encoder-decoder structure. Here is how to build one:

Step 1 — Define a residual block

```
class ResBlock(nn.Module):
    """A single residual block: Conv→BN→ReLU→Conv→BN with skip add."""

    def __init__(self, channels: int) → None:
        super().__init__()
        self.block = nn.Sequential(
            nn.ReflectionPad2d(1), # padding that doesn't create edge
            nn.Conv2d(channels, channels, 3),
            nn.InstanceNorm2d(channels), # InstanceNorm works better than BN for
            style
```

```

        nn.ReLU(inplace=False),          # inplace=False – always safe
        nn.ReflectionPad2d(1),
        nn.Conv2d(channels, channels, 3),
        nn.InstanceNorm2d(channels),
    )

    def forward(self, x: torch.Tensor) → torch.Tensor:
        return x + self.block(x) # residual connection: output = input + learned
delta

```

Step 2 — Build the full generator

```

class ResNetGenerator(BaseGenerator):
    """ResNet generator: downsample → N residual blocks → upsample."""

    def __init__(
        self,
        in_channels: int = 1,
        out_channels: int = 3,
        base_features: int = 64,
        n_blocks: int = 9,          # 9 for 256×256, 6 for 128×128
    ) → None:
        super().__init__()

        # Encoder – downsample to smaller spatial size
        encoder = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(in_channels, base_features, 7),
            nn.InstanceNorm2d(base_features),
            nn.ReLU(inplace=False),
            # Two downsampling steps: 256→128→64
            nn.Conv2d(base_features, base_features * 2, 3, stride=2, padding=1),
            nn.InstanceNorm2d(base_features * 2),
            nn.ReLU(inplace=False),
            nn.Conv2d(base_features * 2, base_features * 4, 3, stride=2, padding=1),
            nn.InstanceNorm2d(base_features * 4),
            nn.ReLU(inplace=False),
        ]

        # Residual blocks – process at the compressed resolution
        res_blocks = [ResBlock(base_features * 4) for _ in range(n_blocks)]

        # Decoder – upsample back to original resolution
        decoder = [
            nn.ConvTranspose2d(base_features * 4, base_features * 2, 3, stride=2,
                              padding=1, output_padding=1),
            nn.InstanceNorm2d(base_features * 2),
            nn.ReLU(inplace=False),
            nn.ConvTranspose2d(base_features * 2, base_features, 3, stride=2,
                              padding=1, output_padding=1),
            nn.InstanceNorm2d(base_features),
            nn.ReLU(inplace=False),
            nn.ReflectionPad2d(3),
            nn.Conv2d(base_features, out_channels, 7),
            nn.Tanh(),
        ]

```

```

        self.model = nn.Sequential(*encoder, *res_blocks, *decoder)
        self._init_weights()

    def _init_weights(self) → None:
        for m in self.modules():
            if isinstance(m, nn.Conv2d) or isinstance(m, nn.ConvTranspose2d):
                nn.init.normal_(m.weight, 0.0, 0.02)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)

    def forward(self, x: torch.Tensor) → torch.Tensor:
        return self.model(x) # type: ignore[no-any-return]

```

Step 3 — Create the config file

Create `configs/model/resnet.yaml`:

```

name: resnet
generator:
  in_channels: 1
  out_channels: 3
  base_features: 64
  n_blocks: 9

discriminator:
  base_features: 64
  n_scales: 3
  spectral_norm: true

```

Step 4 — Export from __init__.py

```

# src/gan_pipeline/models/__init__.py
from .resnet import ResNetGenerator

```

Step 5 — Wire into the training script

In `scripts/train_pix2pix.py`, add a branch for the new model:

```

if cfg.model.name == "unet":
    generator = UNetGenerator(...)
elif cfg.model.name == "resnet":          # ← add this
    generator = ResNetGenerator(
        in_channels=cfg.data.sar_channels,
        out_channels=cfg.data.eo_channels,
        base_features=cfg.model.generator.base_features,
        n_blocks=cfg.model.generator.n_blocks,
    )

```

Step 6 — Write tests

```
# tests/test_pix2pix.py
def test_resnet_generator_output_shape():
    gen = ResNetGenerator(in_channels=1, out_channels=3, base_features=64,
                           n_blocks=6)
    x = torch.randn(1, 1, 256, 256)
    out = gen(x)
    assert out.shape == (1, 3, 256, 256)
    assert out.min() ≥ -1.0 and out.max() ≤ 1.0    # Tanh output in [-1, 1]

def test_resnet_generator_gradients():
    gen = ResNetGenerator(in_channels=1, out_channels=3)
    x = torch.randn(1, 1, 256, 256, requires_grad=True)
    out = gen(x)
    out.mean().backward()
    assert x.grad is not None    # gradients flow all the way back to input
```

Step 7 — Run /smoke-test

```
python scripts/train_pix2pix.py model=resnet training.epochs=2
training.num_workers=0
```

18.3 Adding spectral normalisation to any model

```
import torch.nn as nn

def apply_spectral_norm(module: nn.Module) → nn.Module:
    """Apply spectral norm to all Conv2d layers in a module."""
    for m in module.modules():
        if isinstance(m, nn.Conv2d):
            nn.utils.spectral_norm(m)
    return module

# After building the model:
discriminator = apply_spectral_norm(discriminator)
```


How to Build a Custom Loss Function

This chapter explains how to implement a new loss function and integrate it with the existing loss infrastructure.

19.1 How the existing loss system is structured

The file `models/losses.py` uses a `LossType` enum to select between loss functions:

```
from enum import Enum

class LossType(str, Enum):
    HINGE = "hinge"
    BCE = "bce"
    WGAN = "wasserstein"

def discriminator_loss(
    real_logits: torch.Tensor,
    fake_logits: torch.Tensor,
    loss_type: LossType,
) → torch.Tensor:
    if loss_type == LossType.HINGE:
        return (
            F.relu(1.0 - real_logits).mean()
            + F.relu(1.0 + fake_logits).mean()
        )
    elif loss_type == LossType.BCE:
        ...
    elif loss_type == LossType.WGAN:
        ...
```

19.2 Adding a new adversarial loss — Relativistic Hinge

The Relativistic GAN loss asks "how much more real is the real image than the fake?" rather than just "is this real or fake?"

Step 1 — Add to the enum

```
class LossType(str, Enum):
    HINGE = "hinge"
    BCE = "bce"
```

```
WGAN          = "wasserstein"
REL_HINGE     = "relativistic_hinge"    # ← add this
```

Step 2 — Implement the loss functions

```
def discriminator_loss(real_logits, fake_logits, loss_type):
    ...
    elif loss_type == LossType.REL_HINGE:
        # D wants: real - fake > 1  and  fake - real < -1
        real_relative = real_logits - fake_logits.mean()
        fake_relative = fake_logits - real_logits.mean()
        return (
            F.relu(1.0 - real_relative).mean()
            + F.relu(1.0 + fake_relative).mean()
        )

def generator_loss(fake_logits, loss_type, real_logits=None):
    ...
    elif loss_type == LossType.REL_HINGE:
        # G wants: fake - real > 1
        if real_logits is None:
            raise ValueError("Relativistic hinge needs real_logits in G step")
        fake_relative = fake_logits - real_logits.mean()
        real_relative = real_logits - fake_logits.mean()
        return (
            F.relu(1.0 - fake_relative).mean()
            + F.relu(1.0 + real_relative).mean()
        )
```

Step 3 — Update the trainer to pass real_logits when needed

```
# In pix2pix_trainer.py, generator step:
if self.loss_type == LossType.REL_HINGE:
    # Need real logits for relativistic loss – extra forward pass
    with torch.no_grad():
        real_logits = self.discriminator(cat([sar, real_eo]))
    g_adv = multiscale_generator_loss(
        fake_maps, self.loss_type, real_maps_list=real_logits
    )
else:
    g_adv = multiscale_generator_loss(fake_maps, self.loss_type)
```

Step 4 — Add to the config and test

```
# Use it:
python scripts/train_pix2pix.py training.loss_type=relativistic_hinge

# Test it:
def test_relativistic_hinge_loss():
    real = torch.randn(2, 1, 30, 30)
    fake = torch.randn(2, 1, 30, 30)
```

```
d_loss = discriminator_loss(real, fake, LossType.REL_HINGE)
assert d_loss.shape == ()          # scalar
assert torch.isfinite(d_loss)      # no NaN or Inf
```

19.3 Building a new perceptual loss (LPIPS)

LPIPS (Learned Perceptual Image Patch Similarity) is a perceptual metric that often correlates better with human perception than VGG loss. Here is how to add it:

```
import torch
import torch.nn as nn

class LPIPSLoss(nn.Module):

    def __init__(self) → None:
        super().__init__()
        # pip install lpips
        import lpips
        self.lpips = lpips.LPIPS(net="alex")
        for p in self.parameters():
            p.requires_grad_(False)    # freeze – feature extractor only

    def forward(
        self,
        fake: torch.Tensor,    # (B, 3, H, W), range [-1, 1]
        real: torch.Tensor,
    ) → torch.Tensor:
        # lpips expects [-1, 1] input – matches our normalisation
        return self.lpips(fake, real).mean() # type: ignore[no-any-return]
```

Add `lambda_lpips` to `configs/training/pix2pix.yaml` and integrate in the trainer the same way VGG loss is integrated.

How to Build a New Training Workflow

This chapter explains how the training infrastructure is wired together and how to build an entirely new training workflow from scratch — for example, a CycleGAN unpaired training loop.

20.1 How a trainer is structured

Every trainer follows the same skeleton:

```
class MyTrainer:
    def __init__(self, cfg: DictConfig, device: torch.device) → None:
        # 1. Build models from config
        self.generator = build_generator(cfg)
        self.discriminator = build_discriminator(cfg)

        # 2. Move to device
        self.generator.to(device)
        self.discriminator.to(device)

        # 3. Build optimisers
        self.opt_g = torch.optim.Adam(
            self.generator.parameters(),
            lr=cfg.training.lr_generator,
            betas=(cfg.training.beta1, cfg.training.beta2),
        )
        self.opt_d = torch.optim.Adam(
            self.discriminator.parameters(),
            lr=cfg.training.lr_discriminator,
            betas=(cfg.training.beta1, cfg.training.beta2),
        )

        # 4. Set up output directories
        self.output_dir = Path(cfg.output_dir)
        (self.output_dir / "checkpoints").mkdir(parents=True, exist_ok=True)
        (self.output_dir / "samples").mkdir(parents=True, exist_ok=True)

    def _discriminator_step(self, batch):
        """Update D for one batch. Returns d_loss."""
        ...

    def _generator_step(self, batch):
        """Update G for one batch. Returns losses dict."""
        ...

    def train(self, dataloader) → None:
        mlflow.set_tracking_uri("sqlite:///mlflow.db")
```

```

mlflow.set_experiment(self.cfg.experiment_name)

with mlflow.start_run():
    mlflow.log_params({...})
    for epoch in range(self.cfg.training.epochs):
        for batch in dataloader:
            d_loss = self._discriminator_step(batch)
            g_losses = self._generator_step(batch)
            mlflow.log_metrics({...}, step=epoch)
            if epoch % self.cfg.training.save_every == 0:
                self._save_checkpoint(epoch, {...})

```

20.2 Building a CycleGAN trainer (unpaired)

CycleGAN trains on unpaired images — you don't need matching SAR/EO pairs, just a set of SAR images and a set of EO images. It trains two generators: G_AB (SAR→EO) and G_BA (EO→SAR), enforcing a cycle consistency: $G_BA(G_AB(sar)) \approx sar$.

Step 1 — Create the trainer file

```

# src/gan_pipeline/training/cyclegan_trainer.py

class CycleGANTrainer:

    def __init__(self, cfg: DictConfig, device: torch.device) → None:
        # Two generators, two discriminators
        self.G_AB = ResNetGenerator(...).to(device) # SAR → EO
        self.G_BA = ResNetGenerator(...).to(device) # EO → SAR
        self.D_A = PatchGANDiscriminator(...).to(device) # scores SAR
        self.D_B = PatchGANDiscriminator(...).to(device) # scores EO

        self.lambda_cycle = cfg.training.lambda_cycle # e.g. 10.0
        self.lambda_idt = cfg.training.lambda_identity # e.g. 5.0

        # One optimiser per network pair
        self.opt_G = torch.optim.Adam(
            list(self.G_AB.parameters()) + list(self.G_BA.parameters()),
            lr=cfg.training.lr_generator,
            betas=(cfg.training.beta1, 0.999),
        )
        self.opt_D = torch.optim.Adam(
            list(self.D_A.parameters()) + list(self.D_B.parameters()),
            lr=cfg.training.lr_discriminator,
            betas=(cfg.training.beta1, 0.999),
        )

    def _generator_step(self, real_sar, real_eo):
        # Forward cycle: SAR → fake EO → reconstructed SAR
        fake_eo = self.G_AB(real_sar)
        rec_sar = self.G_BA(fake_eo)

        # Backward cycle: EO → fake SAR → reconstructed EO
        fake_sar = self.G_BA(real_eo)
        rec_eo = self.G_AB(fake_sar)

```

```

# Adversarial: G_AB should fool D_B, G_BA should fool D_A
loss_adv = (
    generator_loss(self.D_B(fake_eo), LossType.HINGE)
    + generator_loss(self.D_A(fake_sar), LossType.HINGE)
)

# Cycle consistency: reconstructed images should match originals
loss_cycle = (
    F.l1_loss(rec_sar, real_sar)
    + F.l1_loss(rec_eo, real_eo)
) * self.lambda_cycle

# Identity: G_AB(real_eo) should ≈ real_eo (colour preservation)
loss_idt = (
    F.l1_loss(self.G_AB(real_eo), real_eo)
    + F.l1_loss(self.G_BA(real_sar), real_sar)
) * self.lambda_idt

loss_G = loss_adv + loss_cycle + loss_idt
self.opt_G.zero_grad()
loss_G.backward()
self.opt_G.step()
return loss_G.item()

```

Step 2 — Create config files

```

# configs/model/cyclegan.yaml
name: cyclegan
generator:
  base_features: 64
  n_blocks: 9
discriminator:
  base_features: 64

# configs/training/cyclegan.yaml
epochs: 200
lr_generator: 0.0002
lr_discriminator: 0.0002
beta1: 0.5
lambda_cycle: 10.0
lambda_identity: 5.0

```

Step 3 — Create the entry point script

```

# scripts/train_cyclegan.py
import hydra
from omegaconf import DictConfig
import torch
from gan_pipeline.training.cyclegan_trainer import CycleGANTrainer
from gan_pipeline.data.paired_dataset import get_paired_data_loader

@hydra.main(config_path="../configs", config_name="config", version_base=None)

```

```
def main(cfg: DictConfig) → None:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    trainer = CycleGANTrainer(cfg, device)
    dataloader = get_paired_dataloader(cfg, split="train")
    trainer.train(dataloader)

if __name__ == "__main__":
    main()
```

Step 4 — Add a Makefile target

```
train-cyclegan:
    python scripts/train_cyclegan.py model=cyclegan training=cyclegan
data=sar_eo
```

How the CI/CD and Pre-commit Were Built

This chapter explains how to set up CI/CD and pre-commit from scratch so you understand every decision made in this project's configuration.

21.1 Setting up pre-commit from scratch

Step 1 — Install pre-commit

```
pip install pre-commit
```

Step 2 — Create .pre-commit-config.yaml

This file defines which hooks to run and how. The project uses `language: system` hooks — they use whatever tools are already installed in your active Python environment rather than downloading separate virtualenvs.

```
repos:
  - repo: local          # run locally installed tools
    hooks:

      - id: ruff
        name: ruff lint
        entry: ruff check # the command to run
        language: system  # use the system/venv installation
        args: [src, tests, scripts]
        pass_filenames: false # pass directories, not individual files
        always_run: true      # run even if no Python files changed

      - id: black
        name: black format check
        entry: black --check # --check = report errors, don't modify files
        language: system
        args: [src, tests, scripts]
        pass_filenames: false
        always_run: true

      - id: isort
        name: isort import check
        entry: isort --check-only
        language: system
        args: [src, tests, scripts]
```



```

    pass_filenames: false
    always_run: true

- id: mypy
  name: mypy type check
  entry: mypy
  language: system
  args: [src]
  pass_filenames: false
  always_run: true

- id: pytest
  name: pytest
  entry: pytest
  language: system
  args: [--no-header, -q]
  pass_filenames: false
  always_run: true

```

Step 3 — Register the hooks

```

pre-commit install
# Output: pre-commit installed at .git/hooks/pre-commit

```

This creates a shell script at `.git/hooks/pre-commit` that git calls before every commit.

Step 4 — Configure tool settings in pyproject.toml

All tool settings are centralised so they agree on line length, Python version, etc.:

```

[tool.black]
line-length = 100
target-version = ["py310"]

[tool.isort]
profile = "black"    # makes isort's style match black's grouping
line_length = 100

[tool.ruff]
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP"] # which rule categories to enforce
ignore = ["N812"]    # specific rule to suppress (lowercase imported as non-
                    lowercase)

[tool.mypy]
python_version = "3.10"
strict = true          # enables all strict checks
ignore_missing_imports = true # don't error on untyped third-party libs

```

21.2 Building GitHub Actions CI from scratch

Step 1 — Create the workflows directory

```
mkdir -p .github/workflows
```

Step 2 — Write pre-commit.yml

```
# .github/workflows/pre-commit.yml
name: pre-commit          # shown in GitHub UI

on:                        # when to trigger
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
  pre-commit:
    runs-on: ubuntu-latest # run on a fresh Ubuntu VM

    steps:
      # 1. Check out the code
      - uses: actions/checkout@v4

      # 2. Set up Python – cache pip downloads for speed
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10"
          cache: pip

      # 3. Install all dependencies (same as local setup)
      - name: Install dev dependencies
        run: pip install -e ".[dev]"

      # 4. Run all hooks against all files
      - name: Run pre-commit hooks
        run: pre-commit run --all-files
```

Step 3 — Write pytest.yml

```
# .github/workflows/pytest.yml
name: pytest

on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
```

```

test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.10"
        cache: pip
    - run: pip install -e ".[dev]"
    - run: pytest

```

Step 4 — Add status badges to README

GitHub generates badge URLs automatically for each workflow:

```

[[pre-commit]](https://github.com/<user>/<repo>/actions/workflows/pre-commit.yml/
badge.svg)]
(https://github.com/<user>/<repo>/actions/workflows/pre-commit.yml)

```

21.3 How MLflow was integrated into the trainers

MLflow integration requires only three additions to a training script:

```

import mlflow

# 1. Set where to store the data (SQLite file in project root)
mlflow.set_tracking_uri("sqlite:///mlflow.db")

# 2. Set the experiment name (creates it if it doesn't exist)
mlflow.set_experiment("my_experiment")

# 3. Wrap the training loop in a run context
with mlflow.start_run():

    # Log hyperparameters once at the start
    mlflow.log_params({
        "loss_type": "hinge",
        "lambda_l1": 100.0,
        "epochs": 200,
    })

    for epoch in range(200):
        # ... training ...

        # Log metrics every epoch
        mlflow.log_metrics({
            "d_loss": d_loss,
            "g_adv": g_adv,
            "g_l1": g_l1,
        }, step=epoch)

```

21.4 Packaging — how pyproject.toml works

The file `pyproject.toml` is the single source of truth for the package. It replaces the older `setup.py`:

```
[project]
name = "gan-pipeline"
version = "0.1.0"
requires-python = "≥3.10"

# Core runtime dependencies
dependencies = [
    "torch≥2.1.0",
    "torchvision≥0.16.0",
    "hydra-core≥1.3.2",
    "mlflow≥2.9.0",
    "loguru≥0.7.2",
    "numpy≥1.24.0",
    "pillow≥10.0.0",
]

# Optional extras — installed with pip install -e ".[dev]" etc.
[project.optional-dependencies]
dev = [
    "pytest≥7.4.0",
    "black≥23.9.0",
    "ruff≥0.1.0",
    "mypy≥1.5.0",
    "isort≥5.12.0",
    "pre-commit≥3.5.0",
]
geo = ["rasterio≥1.3.0", "h5py≥3.9.0"]
eval = ["torch-fidelity≥0.3.0"]

# Tell pip where to find the package source
[tool.setuptools.packages.find]
where = ["src"]
```

Component Integration Map — How Everything Connects

This chapter shows exactly how all the pieces wire together, from the first call to the last.

22.1 Call chain — from CLI to pixel output

```
$ python scripts/train_pix2pix.py training.loss_type=hinge
|
▼
@hydra.main(...)      ← Hydra parses CLI, loads + merges YAML configs
main(cfg: DictConfig)  ← receives one config object
|
|→ torch.device("cuda" or "cpu")
|
|→ UNetGenerator(cfg.model.generator)    ← built from config, moved to device
|→ MultiScaleDiscriminator(cfg.model.discriminator)
|
|→ get_paired_data_loader(cfg, "train")
|    ↳ SideBySidePairedDataset(root, split, image_size, ...)
|        ↳ DataLoader(dataset, batch_size, shuffle, num_workers)
|
|→ Pix2PixTrainer(cfg, device, generator, discriminator)
|
|    ↳ trainer.train(data_loader)
|
|        ↳ mlflow.set_experiment(...)
|
|        ↳ for epoch in range(epochs):
|            for batch in data_loader:    ← {"sar":..., "eo":...}
|
|                ↳ _discriminator_step(batch)
|                    ↳ generator(sar).detach() → fake_eo
|                    ↳ discriminator(cat[sar, real_eo]) → real_maps
|                    ↳ discriminator(cat[sar, fake_eo]) → fake_maps
|                    ↳ multiscale_discriminator_loss(...)
|                    ↳ d_loss.backward()
|                    ↳ opt_d.step()
|
|                ↳ _generator_step(batch)
|                    ↳ generator(sar) → fake_eo
|                    ↳ discriminator(cat[sar, fake_eo]) → fake_maps
|                    ↳ multiscale_generator_loss(...)    g_adv
|                    ↳ F.l1_loss(fake_eo, real_eo)        g_l1
```

```

    |→ VGGPerceptualLoss(fake_eo, eo)    g_vgg
    |→ feature_matching_loss(...)         g_fm
    |→ L_total = g_adv +  $\lambda_1$ *g_l1 +  $\lambda_v$ *g_vgg +  $\lambda_f$ *g_fm
    |→ L_total.backward()
    |→ opt_g.step()

|
|→ mlflow.log_metrics(epoch)
|→ save_checkpoint(epoch)           → outputs/.../checkpoints/
|→ save_image(sample_grid)         → outputs/.../samples/

```

22.2 Data flow — tensor shapes at each stage

```

PNG file (512×256 pixels, uint8 RGB)
|   SideBySidePairedDataset.__getitem__
▼
Split at midpoint:
SAR PIL Image (256×256)    EO PIL Image (256×256)
|                           |
|   Augmentation (same params)   |
|   Resize → 286×286             |
|   RandomCrop → 256×256        |
|   RandomHFlip                 |
▼                               ▼
SAR Tensor (1, 256, 256)    EO Tensor (3, 256, 256)
values in [-1, 1]           values in [-1, 1]
|                           |
└────────── DataLoader ─────────┘
|   batch_size=1
▼
batch["sar"] (1, 1, 256, 256)
batch["eo"]  (1, 3, 256, 256)
|
▼ UNetGenerator
fake_eo (1, 3, 256, 256) values in [-1, 1] ← Tanh activation
|
▼ cat([sar, fake_eo])
disc_input (1, 4, 256, 256) ← 1 SAR channel + 3 EO channels
|
▼ PatchGANDiscriminator (×3 scales)
D0 output (1, 1, 30, 30) ← 900 patch logits at full res
D1 output (1, 1, 14, 14) ← 196 patch logits at half res
D2 output (1, 1, 6, 6) ← 36 patch logits at quarter res
|
▼ multiscale_generator_loss(...)
g_adv scalar ← averaged across all 3 scales

```

22.3 Configuration dependency map

```
configs/config.yaml
├── model: pix2pix      configs/model/pix2pix.yaml
│                       ├── generator.base_features → UNetGenerator.__init__
│                       ├── discriminator.n_scales → MultiScaleDiscriminator.__init__
│                       └── discriminator.spectral_norm → PatchGANDiscriminator.__init__
├── training: pix2pix  configs/training/pix2pix.yaml
│                       ├── lr_generator → Adam(generator.parameters(), lr=...)
│                       ├── loss_type → LossType enum → loss functions
│                       ├── lambda_l1 → g_loss += lambda_l1 * F.l1_loss(...)
│                       ├── lambda_vgg → VGGPerceptualLoss instantiated if > 0
│                       ├── vgg_weights_path → VGGPerceptualLoss(weights_path=...)
│                       └── epochs → for epoch in range(epochs)
└── data: sar_eo       configs/data/sar_eo.yaml
                        ├── root → SideBySidePairedDataset(root=...)
                        ├── image_size → transforms resize/crop target
                        ├── sar_channels → UNetGenerator(in_channels=...)
                        └── dataset_format → get_paired_data_loader() branch
```

22.4 Where each output file comes from

File / Directory	Created by	When
<code>outputs/<name>/</code> <code>checkpoints/epoch_NNNN.pt</code>	<code>utils/checkpointing.py</code> <code>save_checkpoint()</code>	Every <code>save_every</code> epochs
<code>outputs/<name>/samples/</code> <code>epoch_NNNN.png</code>	<code>pix2pix_trainer.py</code> <code>_save_samples()</code>	Every <code>sample_every</code> epochs
<code>outputs/<name>/train.log</code>	Loguru logger in <code>utils/</code> <code>logging.py</code>	Continuous during training
<code>outputs/<name>/hydra/</code> <code>config.yaml</code>	Hydra automatically	Once at training start
<code>mlflow.db</code>	MLflow SQLite backend	Created on first training run, appended to thereafter
<code>data/sar_eo/train/*.png</code>	<code>scripts/prepare_data.py</code>	One-time data preparation step
<code>weights/</code> <code>vgg16-397923af.pth</code>	<code>scripts/</code> <code>download_vgg_weights.py</code>	One-time download

YOU NOW HAVE THE COMPLETE PICTURE

Every component, every connection, every file — you know what it is, why it exists, and how to build it from scratch. Use this knowledge to extend the pipeline confidently: add new models, new losses, new data formats, or entirely new training workflows.

Managing pyproject.toml from the Command Line

THE PROBLEM

Writing and editing `pyproject.toml` by hand is error-prone — wrong indentation, wrong version specifiers, forgetting to add a package to the right extras group. Modern package managers let you manage the entire file from the command line, the same way `npm install` manages `package.json` automatically.

This chapter covers three tools: **uv** (fastest, recommended), **Poetry** (most popular), and **pip-tools** (lightweight). Pick one — they all manage `pyproject.toml` so you never have to edit it by hand for dependency changes.

23.1 uv — the recommended tool

uv is a Rust-based Python package manager from Astral. It is 10–100× faster than pip, manages virtual environments automatically, and can initialise and manage `pyproject.toml` entirely from the CLI.

Install uv

```
# Linux / macOS
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Or via pip
pip install uv
```

Initialise a new project from scratch

```
uv init gan-pipeline
cd gan-pipeline
```

This creates:

```
gan-pipeline/  
├─ pyproject.toml      ← auto-generated with sane defaults  
├─ README.md  
├─ .python-version     ← pins the Python version (e.g. 3.10)  
└─ src/  
    └─ gan_pipeline/  
        └─ __init__.py
```

The generated `pyproject.toml` looks like:

```
[project]  
name = "gan-pipeline"  
version = "0.1.0"  
requires-python = "≥3.10"  
dependencies = []      ← uv manages this list
```

Add core dependencies

```
# Add one package — uv resolves, installs, AND writes it to pyproject.toml  
uv add torch torchvision  
uv add hydra-core omegaconf mlflow loguru numpy pillow scipy tqdm
```

Each `uv add` command:

- Resolves the latest compatible version
- Installs it into the virtual environment
- Writes it to the `dependencies` list in `pyproject.toml`
- Updates `uv.lock` (like `package-lock.json` — pins exact versions)

Add dev dependencies (extras group)

```
# These go into [project.optional-dependencies] dev = [...]  
uv add --dev pytest pytest-cov black ruff isort mypy types-Pillow pre-commit
```

Add optional extras groups

```
# Creates [project.optional-dependencies] geo = [...]  
uv add --optional geo rasterio h5py  
  
# Creates [project.optional-dependencies] eval = [...]  
uv add --optional eval torch-fidelity
```

Remove a dependency

```
uv remove scipy          # removes from dependencies + uninstalls
uv remove --dev pytest-cov # removes from dev group
```

Upgrade a dependency

```
uv add torch --upgrade    # upgrade to latest compatible version
uv lock --upgrade         # upgrade everything in uv.lock
```

Install from existing pyproject.toml (on a new machine)

```
uv sync          # installs all dependencies including dev
uv sync --no-dev # core only (for production)
uv sync --extra geo # core + geo extras
```

Run commands inside the managed environment

```
uv run pytest          # run pytest without activating manually
uv run python scripts/train_pix2pix.py
uv run make lint
```

UV REPLACES BOTH PIP AND VENV

You do not need `python -m venv .venv` or `pip install` when using uv. It manages the virtual environment automatically. The Makefile's `make venv` and `make install` targets can be replaced by `uv sync`.

Complete uv workflow for this project from scratch

```
# 1. Install uv once
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Initialise project
uv init gan-pipeline && cd gan-pipeline

# 3. Add all dependencies
uv add torch torchvision hydra-core omegaconf mlflow loguru numpy pillow scipy tqdm
uv add --dev pytest pytest-cov black ruff isort mypy types-Pillow pre-commit
uv add --optional geo rasterio h5py
uv add --optional eval torch-fidelity

# 4. Install and activate
uv sync --extra geo --extra eval
source .venv/bin/activate
```

```
# 5. Install pre-commit hooks
pre-commit install

# Done – pyproject.toml was written entirely by uv commands
```

23.2 Poetry — the most widely used tool

Poetry has been the industry standard for Python project management since 2018. It manages `pyproject.toml`, virtual environments, and publishing.

Install Poetry

```
curl -sSL https://install.python-poetry.org | python3 -
```

Initialise a new project

```
poetry new gan-pipeline      # creates full structure
# or, inside an existing directory:
poetry init                  # interactive wizard that builds pyproject.toml
```

The `poetry init` wizard asks you:

```
Package name [gan-pipeline]:
Version [0.1.0]:
Description []: Production-grade SAR→EO GAN pipeline
Compatible Python versions [^3.10]: ≥3.10
Would you like to define your main dependencies? yes
  → torch, torchvision, hydra-core, ...
Would you like to define your dev dependencies? yes
  → pytest, black, ruff, ...
```

Add dependencies

```
# Core dependency
poetry add torch torchvision hydra-core omegaconf mlflow loguru

# Dev dependency (added to [tool.poetry.dev-dependencies])
poetry add --group dev pytest black ruff isort mypy pre-commit

# Optional extras group
poetry add --optional rasterio h5py      # for geo extra
```

Remove a dependency

```
poetry remove scipy
poetry remove --group dev pytest-cov
```

Upgrade dependencies

```
poetry update          # upgrade all to latest compatible
poetry update torch    # upgrade one package
```

Install on a new machine

```
poetry install          # installs everything + dev
poetry install --no-dev # core only
```

Run commands

```
poetry run pytest
poetry run python scripts/train_pix2pix.py
poetry shell          # activate the managed venv in a subshell
```

POETRY VS PYPROJECT.TOML STANDARD

Poetry uses its own `[tool.poetry]` section in `pyproject.toml` in addition to the standard `[project]` section. This means a Poetry-managed `pyproject.toml` is slightly different from one managed by `uv` or `setuptools`. If you use Poetry, stick with it consistently across the project.

23.3 pip-tools — lightweight pinning without a full package manager

If you want to keep plain `pip` but get reproducible builds with pinned versions, `pip-tools` is the answer. It compiles `pyproject.toml` into a locked `requirements.txt`.

Install pip-tools

```
pip install pip-tools
```

Compile requirements from pyproject.toml

```
# Generates requirements.txt with pinned versions from [project].dependencies
pip-compile pyproject.toml -o requirements.txt

# Generates dev-requirements.txt from [project.optional-dependencies].dev
pip-compile pyproject.toml --extra dev -o requirements-dev.txt
```

Install pinned requirements

```
pip-sync requirements-dev.txt # installs exactly what's in the lock file
```

Upgrade a pinned package

```
pip-compile --upgrade-package torch pyproject.toml -o requirements.txt
```

WHEN TO USE PIP-TOOLS

Use pip-tools when you want to keep pip as your installer but need reproducible builds (same exact versions on every machine). It is lighter than Poetry or uv but does not manage virtual environments — you still create and activate the venv manually.

23.4 Comparison table

Feature	uv	Poetry	pip-tools + pip
Speed	★★★★★ (Rust, very fast)	★★★★ (Python, moderate)	★★★ (depends on pip)
Manages venv automatically	Yes	Yes	No — manual
Writes pyproject.toml via CLI	Yes — <code>uv add</code>	Yes — <code>poetry add</code>	No — edit by hand, then compile
Lock file	<code>uv.lock</code>	<code>poetry.lock</code>	<code>requirements.txt</code>
Compatible with standard pip	Yes	Mostly	Yes
Extras groups (<code>[dev]</code> , <code>[geo]</code>)	Yes — <code>uv add --optional</code>	Yes — <code>poetry add -- group</code>	Yes — <code>pip-compile -- extra</code>
Learning curve	Low	Medium	Low

Best for

New projects,
speed priority

Teams familiar with
Poetry ecosystem

Projects already using
pip

23.5 Applying this to the existing project

The gan-pipeline project currently uses plain `pip install -e ".[dev]"`. To switch to uv without changing anything else:

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Clone the repo (pyproject.toml already exists)
git clone https://github.com/audreyr/gan-pipeline.git
cd gan-pipeline

# uv reads the existing pyproject.toml and installs everything
uv sync --extra geo --extra eval

# Activate
source .venv/bin/activate

# Install pre-commit hooks
pre-commit install

# Download VGG weights
make download-weights
```

From this point, use `uv add <package>` to add new dependencies instead of editing `pyproject.toml` by hand. The file is updated automatically.

Adding a new dependency to this project (uv workflow)

```
# Adding albumentations for advanced augmentation
uv add albumentations

# Adding it to geo extras instead of core
uv add --optional geo albumentations

# The pyproject.toml geo section now automatically includes albumentations:
# geo = ["rasterio>=1.3.0", "h5py>=3.9.0", "albumentations>=1.3.0"]
```

RECOMMENDATION FOR THIS PROJECT

Use

UV

for all new dependency management. It is the fastest tool available, reads the existing `pyproject.toml` without any changes, and replaces both `python -m venv` and `pip`

`install` with a single `uv sync` command. The rest of the project (Makefile, CI, pre-commit) works unchanged.

Managing CI/CD from the Command Line

THE GOAL

Every CI/CD action — trigger a workflow, check run status, view logs, manage secrets, test workflows locally — can be done from the terminal without opening a browser. This chapter covers the three tools that make this possible: the

GITHUB CLI (GH)

,

ACT

(local Actions runner), and the

PRE-COMMIT CLI

.

24.1 GitHub CLI — gh

gh is GitHub's official command-line interface. It communicates directly with the GitHub API, so every action you would take in the web UI can be scripted.

Install

```
# Ubuntu / Debian
sudo apt install gh

# macOS
brew install gh

# Windows (winget)
winget install GitHub.cli

# Authenticate (one-time)
gh auth login          # prompts: GitHub.com → HTTPS → browser/token
```

View workflow runs

```
# List the most recent runs across all workflows
gh run list

# Filter by workflow file
gh run list --workflow pre-commit.yml
```

```
gh run list --workflow pytest.yml

# Show status of a specific run (replace with run ID from list)
gh run view 12345678

# Watch a run in real time (streams logs until completion)
gh run watch 12345678
```

View logs

```
# Full logs for a run
gh run view 12345678 --log

# Only failed step logs
gh run view 12345678 --log-failed
```

Trigger a workflow manually

```
# Run a workflow on demand (requires workflow_dispatch trigger in the YAML)
gh workflow run pre-commit.yml
gh workflow run pytest.yml --ref master

# Pass inputs to a parametrised workflow
gh workflow run train.yml --field epochs=50 --field loss_type=hinge
```

To enable manual triggering, add `workflow_dispatch:` to your workflow YAML:

```
on:
  push:
    branches: [master]
  pull_request:
    branches: [master]
  workflow_dispatch:      # ← adds "Run workflow" button + gh workflow run support
  inputs:
    epochs:
      description: Number of training epochs
      default: "10"
```

Re-run a failed workflow

```
# Re-run all jobs in a failed run
gh run rerun 12345678

# Re-run only failed jobs (skips passing jobs – faster)
gh run rerun 12345678 --failed
```

Download workflow artifacts

```
# List artifacts produced by a run
gh run download 12345678 --dry-run

# Download all artifacts into ./artifacts/
gh run download 12345678 --dir artifacts/

# Download a specific named artifact
gh run download 12345678 --name coverage-report
```

Manage secrets from the CLI

```
# Set a repository secret (prompts for value securely)
gh secret set WANDB_API_KEY
gh secret set HF_TOKEN --body "hf_abc123xyz"

# List secrets (names only — values are never shown)
gh secret list

# Delete a secret
gh secret delete WANDB_API_KEY
```

SECRETS IN THIS PROJECT

This project currently needs no secrets for CI — tests run on CPU with synthetic data. If you add cloud storage (S3, GCS) or experiment tracking (W&B), add the keys with `gh secret set` and reference them in the workflow YAML as `${{ secrets.KEY_NAME }}`.

Manage workflow files

```
# List all workflows in the repo
gh workflow list

# Disable a workflow (stops it from triggering)
gh workflow disable pytest.yml

# Re-enable it
gh workflow enable pytest.yml

# View the YAML of a workflow without opening the file
gh workflow view pre-commit.yml
```

Open the run in a browser (when you need the visual diff)

```
gh run view 12345678 --web      # opens in browser
gh repo view --web             # open the repo home page
```

24.2 act — Run GitHub Actions Locally

`act` is an open-source tool that runs GitHub Actions workflows on your machine using Docker. It lets you test workflow changes without pushing to GitHub and waiting for a runner.

Install act

```
# macOS
brew install act

# Linux (via release binary)
curl -s https://raw.githubusercontent.com/nektos/act/master/install.sh | sudo bash

# or via Go
go install github.com/nektos/act@latest
```

`act` requires Docker to be running. It pulls GitHub-runner-compatible Docker images on first use.

Run all workflows locally

```
# Simulate a push event (what runs on git push)
act push

# Simulate a pull_request event
act pull_request

# Run a specific workflow
act push --workflows .github/workflows/pytest.yml

# Run a specific job inside a workflow
act push --job run-tests
```

Choose the runner image

```
# Micro image — fast but minimal (good for pure-Python, no Docker-in-Docker)
act push -P ubuntu-latest=cattthehacker/ubuntu:act-latest

# Full image — matches GitHub's hosted runner (large, ~18 GB)
act push -P ubuntu-latest=ghcr.io/cattthehacker/ubuntu:full-latest
```

For this project the micro image is sufficient — all CI needs is Python and pip.

Pass secrets to act

```
# Inline
act push --secret WANDB_API_KEY=abc123

# From a file (one KEY=value per line, like .env)
act push --secret-file .secrets
```

Dry run — see what would execute without running

```
act push --dryrun
```

Typical act workflow for this project

```
# 1. Edit a workflow file
vim .github/workflows/pytest.yml

# 2. Test it locally before pushing
act push --workflows .github/workflows/pytest.yml -P ubuntu-latest=catthehacker/
ubuntu:act-latest

# 3. If it passes locally, push
git add .github/workflows/pytest.yml
git commit -m "Update pytest workflow"
git push
```

ACT LIMITATIONS

- Services (Postgres, Redis) defined under `services:` may not start correctly in all images
- Actions that use `GITHUB_TOKEN` for write operations (create release, deploy) need `--actor` and a real token
- GPU runners are not available locally through act — skip GPU-dependent jobs with an `if:` condition during local testing

24.3 Pre-commit CLI — Full Control Over Hooks

Pre-commit has a rich CLI beyond `pre-commit install`. You can run individual hooks, update hook versions, and audit what hooks are configured — all without committing.

Run all hooks against all files

```
pre-commit run --all-files
```

Run a specific hook only

```
pre-commit run ruff --all-files
pre-commit run black --all-files
pre-commit run mypy --all-files
pre-commit run pytest --all-files
```

Run hooks against staged files only (same as what git commit triggers)

```
pre-commit run
```

Update hook versions to latest

```
# Update all hooks to their latest tagged versions
pre-commit autoupdate

# The .pre-commit-config.yaml rev: fields are rewritten automatically
# Review the diff, then commit:
git diff .pre-commit-config.yaml
git add .pre-commit-config.yaml
git commit -m "Update pre-commit hook versions"
```

Show installed hooks and their status

```
pre-commit run --show-diff-on-failure --all-files
```

Clean the pre-commit cache (if hooks behave strangely)

```
pre-commit clean          # removes cached hook environments
pre-commit run --all-files # re-installs and runs
```

Temporarily skip a specific hook

```
# Skip one hook by its id
SKIP=mypy git commit -m "wip: skip typecheck this once"
```

```
# Skip multiple hooks
SKIP=mypy,pytest git commit -m "wip: skip slow checks"
```

PROJECT POLICY

`--no-verify` skips ALL hooks and is forbidden by this project's `/commit` skill. Use `SKIP=hook-id` when you genuinely need to bypass one hook — it is explicit and auditable.

Uninstall / reinstall hooks

```
pre-commit uninstall      # removes the git hook scripts
pre-commit install        # reinstalls them from .pre-commit-config.yaml
```

24.4 Workflow configuration reference for this project

The two workflow files live in `.github/workflows/`. Here is what each flag does:

```
# .github/workflows/pre-commit.yml
name: pre-commit
on:
  push:
    branches: [master]      # runs on every push to master
  pull_request:
    branches: [master]      # runs on every PR targeting master

jobs:
  pre-commit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10" # pins the Python version
          cache: pip             # caches pip downloads between runs
      - name: Install deps
        run: pip install -e ".[dev]" # installs the package + dev extras
      - name: Run pre-commit
        run: pre-commit run --all-files # runs all 5 hooks
```

```
# .github/workflows/pytest.yml
name: pytest
on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
  test:
```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: "3.10"
      cache: pip
  - name: Install deps
    run: pip install -e ".[dev]"
  - name: Run tests
    run: pytest --no-header -q      # same flags as the pre-commit hook

```

Add a caching layer for faster runs

```

- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('pyproject.toml') }}
    restore-keys: |
      ${{ runner.os }}-pip-

```

Insert this step before `Install deps`. The cache key changes whenever `pyproject.toml` changes, so stale environments are never reused.

Add a test matrix (test multiple Python versions)

```

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
    steps:
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

```

24.5 Quick-reference command card

Task	Command
See recent CI runs	<code>gh run list</code>
Watch a run live	<code>gh run watch <id></code>
View failed logs	<code>gh run view <id> --log-failed</code>
Re-run only failed jobs	<code>gh run rerun <id> --failed</code>

Trigger workflow manually	<code>gh workflow run pytest.yml</code>
Set a secret	<code>gh secret set KEY_NAME</code>
Test workflow locally	<code>act push --workflows .github/workflows/pytest.yml</code>
Run all pre-commit hooks	<code>pre-commit run --all-files</code>
Run one hook only	<code>pre-commit run mypy --all-files</code>
Skip one hook on commit	<code>SKIP=mypy git commit -m "..."</code>
Update hook versions	<code>pre-commit autoupdate</code>
Download CI artifacts	<code>gh run download <id> --dir artifacts/</code>
Disable a workflow	<code>gh workflow disable pytest.yml</code>
Open run in browser	<code>gh run view <id> --web</code>